Day 23: Search Payee by Name

1. Day Number

Day 23

2. Topic Name

Searching in a List of Dictionaries

3. Connection

Earlier, you created a **payee list** using a list of dictionaries.

Today, you will learn how to:

- Ask the user for a payee name
- Search through the payee records
- Display the matching payee
- Show a message when the payee is unavailable

---

4. Important Topics

Loop

A loop checks each payee record one by one.

```
for payee in payees:
    # Check the current payee
```

Dictionary access

Each payee is a dictionary containing details such as name and account number.

```
payee["name"]
payee["account_number"]
```

Boolean flag

A Boolean flag remembers whether the payee was found.

```
found = False
```

When a matching payee is found:

```
found = True
```

A Boolean value can only be:

- `True`
- `False`

---

5. Foundational Notes

List of dictionaries

A list can hold multiple dictionaries.

```
people = [
    {"name": "Amit", "city": "Delhi"},
    {"name": "Neha", "city": "Pune"}
]
```

Here:

- `people` is a list
- Each item is a dictionary
- Each dictionary represents one person

Searching

Searching means checking every record until a match is found.

```
Check first record
    ↓
Is it the requested name?
    ↓
No → Check next record
    ↓
Yes → Display the record
```

Case-insensitive searching

Python treats these as different strings:

```
Amit
amit
AMIT
```

You can use `.lower()` to compare them fairly:

```
entered_name.lower() == stored_name.lower()
```

Why use a Boolean flag?

Suppose the payee list has three records. If no record matches, you need to print `"Payee not found"` only once.

A flag helps remember the final result:

```
found = False
```

If a match is found:

```
found = True
```

After the loop, check the flag.

6. Easy Example

This example searches for a student inside a list of dictionaries:

```
students = [
    {"name": "Ravi", "course": "Python"},
    {"name": "Meera", "course": "SQL"}
]

search_name = "Meera"
found = False

for student in students:
    if student["name"] == search_name:
        print(student)
        found = True
        break

if found == False:
    print("Student not found")
```

What happens?

1. `found` starts as `False`.
2. The loop checks each student.
3. When `"Meera"` matches, the dictionary is printed.
4. `found` becomes `True`.
5. `break` stops the loop because the required student has been found.

This example demonstrates the pattern. Your practice problem should be written separately.

---

7. Problem Statement

Create a list containing multiple payees.

Each payee should be represented by a dictionary containing details such as:

- Payee name
- Account number

Ask the user to enter the name of the payee they want to search for.

Search through the payee list:

- If the payee is found, print the payee's details.
- Otherwise, print `"Payee not found"`.

---

8. Concepts Used

- List
- Dictionary
- List of dictionaries
- `input()`

- `for` loop
- Dictionary key access
- `if` condition
- Boolean flag
- String comparison
- `.strip()`
- `.lower()`
- `break`
- Function
- Parameter
- Return value

---

9. Thought Process

Step 1: Create the payee records

Create a list containing a few payee dictionaries.

Each dictionary can contain:

```
name
account_number
```

Step 2: Ask for the payee name

Take the search name from the user.

Clean the input using `.strip()` so that extra spaces do not affect the search.

Step 3: Create a Boolean flag

Initially, assume that the payee has not been found.

```
found = False
```

Step 4: Loop through the list

Use a `for` loop to inspect each payee dictionary.

Step 5: Compare the names

Compare the entered name with the name stored in the current dictionary.

Using `.lower()` will make the search case-insensitive.

Step 6: Handle a successful match

When the names match:

- Print or return the payee details
- Change the flag to `True`
- Stop the loop using `break`

Step 7: Handle an unsuccessful search

After the loop, check the flag.

If it is still `False`, the payee was not found.

---

10. Pseudocode

```
START

CREATE a list of payee dictionaries

DEFINE a function search_payee that receives:
    payee list
    search name

    SET found to False

    FOR each payee in payee list

        IF current payee name matches search name
            SET found to True
            RETURN current payee

    RETURN no result

ASK user to enter payee name

CLEAN the entered name

CALL search_payee function

IF function returned a payee
    PRINT payee details
ELSE
    PRINT "Payee not found"

END
```

---

11. Suggested Solving Approach: Functional Approach

Create a function responsible only for searching.

Possible function structure:

```
def search_payee(payee_list, search_name):
    # Search here
    # Return the matching dictionary
    # Return None when no match exists
```

Then call the function from the main program:

```
result = search_payee(payees, entered_name)
```

Finally, check the returned value:

```
If result contains a payee:
    Print its details
Otherwise:
    Print not found
```

Why use a function?

A function makes the program:

- Easier to read
- Easier to test
- Easier to reuse
- Easier to extend later

For example, the same function could later be used in a bank menu.

---

12. Easy Edge Cases

Payee not found

The user enters a name that does not exist.

```
Enter payee name: Mohan
Payee not found
```

Make sure the not-found message appears only once, after the complete list has been checked.

Empty payee list

```
payees = []
```

The loop will not execute because there are no records.

The program should display:

```
Payee not found
```

Different uppercase or lowercase letters

Stored name:

```
Rahul
```

Entered name:

```
rahul
```

Using `.lower()` during comparison allows this to match.

Extra spaces

Input:

```
   Rahul
```

Using `.strip()` removes the unnecessary spaces.

Duplicate names

Two payees might have the same name. For today's simple problem, you can return the first matching payee by using `break` or an early `return`.

---

13. Expected Input

Example payee list:

```
Amit – Account 1001
Neha – Account 1002
Ravi – Account 1003
```

User input:

```
Enter payee name: Neha
```

Another possible input:

```
Enter payee name: Karan
```

---

14. Expected Output

When the payee is found

```
Enter payee name: Neha

Payee found
Name: Neha
Account number: 1002
```

When the payee is not found

```
Enter payee name: Karan

Payee not found
```

When the list is empty

```
Enter payee name: Neha

Payee not found
```

---

15. Hint Only

Inside the function:

1. Start with `found = False`.

2. Loop through every payee dictionary.
3. Access the name using the dictionary key.
4. Compare both names after applying `.strip().lower()`.
5. When they match, return the current payee dictionary.
6. If the loop finishes without a match, return `None`.

Think about this condition:

```
if payee["name"].lower() == search_name.lower():
```

Do not print `"Payee not found"` inside the loop, because the current record may not match while a later record might match.